❸ Values can be written from C# objects to the HTML.

❹ C# constructs such as for loops can be used in Razor.

❺ Tag Helpers can also be used outside forms to help in other HTML generation.

This example demonstrates a variety of Razor features. There's a mixture of HTML that's written unmodified to the response output, and there are various C# constructs used to generate HTML dynamically. In addition, you can see several Tag Helpers. These look like normal HTML attributes that start with `asp-`, but they're part of the Razor language. They can customize the HTML element they're attached to, changing how it's rendered. They make building HTML forms much simpler than they would be otherwise. Don't worry if this template is a bit overwhelming at the moment; we'll break it all down as you progress through this chapter and the next.

Razor Pages are compiled when you build your application. Behind the scenes, they become another C# class in your application. It's also possible to enable runtime compilation of your Razor Pages. This allows you to modify your Razor Pages while your app is running without having to explicitly stop and rebuild. This can be handy when developing locally, but it's best avoided when you deploy to production. You can read how to enable this at http://mng.bz/jP2P.

NOTE As with most things in ASP.NET Core, it's possible to swap out the Razor templating engine and replace it with your own server-side rendering engine. You can't replace Razor with a client-side framework like Angular or React. If you want to take this approach, you'd use minimal APIs or web API controllers instead and a separate client-side framework.

In the next section we'll look in more detail at how Razor views fit into the Razor Pages framework and how you can pass data from your Razor Page handlers to the Razor view to help build the HTML response.